Manual técnico — Proyecto Oráculo (ADAG1)

Autor: Juan Felipe Henao · **Versión:** primera entrega (solución base), rama `main`

Este manual explica cómo está construida la solución base y cómo reproducir su ejecución sin ayuda de su autor.

1. Entorno

Elemento	Valor
Plataforma de ejecución	Google Colab, entorno con GPU **NVIDIA T4** (16 GB)
Lenguaje	Python 3 (la versión que trae Colab; probado con 3.13)
Modelo	`Qwen/Qwen3-1.7B` (alias `qwen17b`), fp16, ≈ 3,4 GB de VRAM
Dependencias	`transformers`, `accelerate`, `bitsandbytes>=0.46.1`, `nltk`, `spacy` (+ `en_core_web_sm`), `emoji`, `langdetect`, `immutabledict`, `pandas`, `matplotlib`; repositorio `allenai/open-instruct` (verificador)
Archivos del curso	`oraculo.py`, `ayudas.py`, `datos_visibles.json`: están en la raíz del fork, y la celda 1 de cada notebook descarga la misma versión desde el repositorio del curso para que funcione en Colab
Almacenamiento opcional	Google Drive, para el caché del oráculo y el registro
Pruebas sin GPU	Python 3.9 o superior, solo biblioteca estándar; cualquier sistema operativo

La celda 1 de cada notebook instala todo. Después de ejecutarla hay que **reiniciar la sesión**, porque si no `transformers` puede quedar cargada a medias.

2. Arquitectura

El repositorio es un fork de JuManoel/ADA-2026-2-GP01. Los archivos del curso se conservan sin cambios en la raíz, y la solución se agrega en carpetas propias:

```
ADA-2026-2-GP01/  (fork)
├── README.md                     descripción de la solución (este proyecto)
├── README_curso.md               README original del curso
├── oraculo.py                    del curso (caja negra, sin modificar)
├── ayudas.py                     del curso (sin modificar)
├── datos_visibles.json           del curso (sin modificar)
├── proyecto_oraculo.ipynb        notebook plantilla del curso (sin modificar)
├── entrega.json                  salida final: configuración ganadora
├── notebooks/
│   ├── notebook_busqueda.ipynb   búsqueda (algoritmo) + evidencia
│   └── notebook_evaluacion.ipynb evaluación de entrega.json
├── resultados/registro_busqueda.csv
├── pruebas/
```

```
│   ├── verificar_entrega.py
│   └── prueba_sintetica.py
└── docs/
    ├── manual_tecnico.pdf
    └── manual_tecnico.md
```

Componente	Responsabilidad
`oraculo.py` (del curso, caja negra)	`Oraculo.evaluar(config, instancias, semilla)` genera las respuestas del modelo y las verifica; `Oraculo.validar(config, n)` mide sobre la partición de validación; `espacio(temps)` lista las configuraciones válidas. Guarda un caché de respuestas.
`ayudas.py` (del curso)	Cargar el modelo y los datos, dividir búsqueda y validación, `ver_prompt`, `curva`, `entrega`.
`notebook_busqueda.ipynb`	Implementa el ascenso por coordenadas guiado por trazas, registra cada evaluación, valida y exporta `entrega.json`.
`notebook_evaluacion.ipynb`	Lee `entrega.json`, localiza los datos de prueba y reporta la precisión.
`registro_busqueda.csv`	Evidencia de todas las evaluaciones de la búsqueda.
`verificar_entrega.py`	Comprueba que `entrega.json` sea válido y coincida con la mejor fila del registro.
`prueba_sintetica.py`	Ejecuta la misma lógica de búsqueda contra un oráculo sintético con resultado conocido.

Secciones del notebook de búsqueda

Sección	Qué hace
1	Instala dependencias y descarga los archivos del curso.
2	Parámetros (`MODELO`, `SEMILLA`, `MAX_PASADAS`, `UMBRAL_TRAZA`, `PROBAR_TEMPERATURAS`, `USAR_DRIVE`, `GRUPO`, `SEMANA`), carga del modelo, datos y oráculo con caché.
3	Detecta ranuras y opciones desde `espacio()`; `config_con` crea vecinos que difieren en una ranura.
4	`evaluar_registrar`: evalúa, cuenta fallos por familia y guarda una fila en el CSV.
5	Evalúa la configuración inicial y mide el tiempo por evaluación.
6	Cuenta fallos por familia y decide el orden de ranuras con la hipótesis ranura → familias.
7	Ascenso por coordenadas con poda por trazas.
8	Prueba las temperaturas 0,3 y 0,7.
9	Tabla de evaluaciones, curva de mejor precisión, costo y gráfica por ranura.
10	Opciones descartadas, mapa observado ranura → familias y fallos que persisten.
11	`oraculo.validar` sobre las 87 instancias de validación.
12	Escribe y descarga `entrega.json` y el registro.

3. Funcionamiento

3.1 Flujo del programa

1. Cargar modelo, datos (100 instancias de búsqueda y 87 de validación) y oráculo.
2. Evaluar la configuración inicial c_0 = (0, 0, 0, 0, 0), temperatura 0,0.
3. Ordenar las ranuras según los fallos que las trazas de c_0 asocian a cada una.
4. Repetir pasadas de ascenso por coordenadas hasta convergencia o `MAX_PASADAS`.
5. Ajustar la temperatura.
6. Validar y exportar.

3.2 Algoritmo: ascenso por coordenadas guiado por trazas

```
actual ← c₀;  (p, F) ← evaluar(actual)            # F: fallos por familia
orden ← ranuras ordenadas por fallos asociados en F
descartadas[r] ← ∅ para cada ranura r
para pasada = 1 .. MAX_PASADAS:
    mejoró ← falso
    para r en orden:
        (p_base, F_base) ← (p, F);  mejor ← (actual[r], p, F)
        para cada opción o ≠ actual[r], o ∉ descartadas[r]:
            (p_o, F_o) ← evaluar(actual con r = o)
            si existe familia f con F_o[f] − F_base[f] ≥ UMBRAL_TRAZA y p_o ≤ p_base:
                descartadas[r] ← descartadas[r] ∪ {o}
            si p_o > mejor.p:  mejor ← (o, p_o, F_o)
        si mejor.o ≠ actual[r]:
            actual[r] ← mejor.o;  (p, F) ← (mejor.p, mejor.F);  mejoró ← verdadero
    si no mejoró:  parar        # óptimo local: ningún cambio de una ranura mejora
probar temperaturas 0,3 y 0,7 con las ranuras fijas
```

En empate se conserva la opción actual, para no moverse por diferencias de ruido.

3.3 Estructuras de datos

Estructura	Tipo	Contenido
Configuración	`dict`	`{"rol": int, "estrategia": int, "formato": int, "estilo": int, "cierre": int, "temperatura": float}`
`INDICE`	`dict[tupla → config]`	Acceso O(1) a la configuración oficial a partir de sus índices
`REGISTRO`	`list[dict]`	Una fila por evaluación; se exporta a CSV
`RESULTADOS`	`dict[tupla → (precisión, Counter)]`	Precisión y fallos por familia de cada configuración evaluada
`DESCARTADAS`	`dict[ranura → dict[opción → motivo]]`	Opciones podadas y la familia que las descartó
Fallos por familia	`collections.Counter`	Familia (prefijo de `violo`, por ejemplo `startend`) → número de instancias

3.4 Condición de parada y costo

- Se detiene cuando una pasada completa no mejora la mejor precisión, o al llegar a `MAX_PASADAS` = 3.
- Costo máximo en consultas: n · [1 + I · Σ(k_p − 1) + (|T| − 1)] = 100 · (1 + 3 · 35 + 2) = **10.800**, es decir, O(I · m · k · n). El caché del oráculo y la poda lo reducen: la corrida real usó 60 configuraciones distintas.

4. Uso

4.1 Evaluar `entrega.json` (notebook de evaluación)

1. Abrir el notebook en Colab y elegir **T4 GPU**.
2. Ejecutar la celda 1 y luego **Entorno de ejecución ▸ Reiniciar sesión**.
3. Opcional: arrastrar al panel de archivos de Colab el archivo de instancias de prueba (`.json`, mismo formato de `datos_visibles.json`).
4. **Entorno de ejecución ▸ Ejecutar todas.**

Formato de entrada. `entrega.json`, local o descargado de este repositorio en GitHub:

```
        {"grupo": "G01", "config": {"rol": 0, "estrategia": 0, "formato": 0, "estilo": 5,
"cierre": 1, "temperatura": 0.0}, "semana": 5}
```

Y un archivo de instancias: una lista de objetos con al menos `prompt` e `ids` (y `kwargs` para el verificador). Las instancias marcadas `"split": "descartada"` se ignoran.

Ejemplo de salida:

```
Fuente:        partición de validación pública (demostración)
Instancias:    87
Configuración: {'rol': 0, 'estrategia': 0, 'formato': 0, 'estilo': 5, 'cierre': 1}
temperatura 0.0
PRECISIÓN:     10.3%
```

Significado. La precisión es la fracción de instancias en las que la respuesta cumplió todas sus restricciones (todo o nada). La tabla de fallos indica la primera restricción violada en cada instancia fallida, agrupada por familia. El resultado se guarda en `resultado_evaluacion.json`.

4.2 Reproducir la búsqueda (notebook de búsqueda)

1. Abrir el notebook en Colab con **T4 GPU** y ejecutar la celda 1; reiniciar la sesión.
2. Ejecutar la celda 2 y aceptar el acceso a Google Drive. Hay que marcar todas las casillas de permisos.
3. Ejecutar las secciones en orden. Cada evaluación nueva tarda unos 205 s; las que están en caché, 0 s.
4. Al final se descargan `entrega.json` y `registro_busqueda.csv`.

4.3 Pruebas sin GPU

```
        python pruebas/verificar_entrega.py
```

```
python pruebas/prueba_sintetica.py
```

5. Validación y límites

5.1 Pruebas disponibles

Prueba	Qué verifica	Resultado esperado
`verificar_entrega.py`	Claves y tipos de `entrega.json`; índices 0–7; temperatura en {0,0; 0,3; 0,7}; coincidencia con la mejor fila del registro	`OK: entrega.json es válido y coincide con la mejor configuración del registro.`
`prueba_sintetica.py`	Con un oráculo sintético de ranuras independientes (semilla 7): la mejor precisión nunca baja, no se supera la cota $1 + l \cdot \Sigma(k_p - 1)$ y se encuentra el óptimo global	`Pasadas: 2  Evaluaciones: 64  Cota teórica: 106` y `OK: las 3 verificaciones pasaron.`
Sección 11 del notebook de búsqueda	Generalización a 87 instancias no usadas en la búsqueda	10,3 %

5.2 Errores conocidos y soluciones

Síntoma	Causa	Solución
`MessageError: credential propagation was unsuccessful` al montar Drive	Permisos de Drive no concedidos o ventana emergente bloqueada	Volver a ejecutar y marcar todas las casillas, montar Drive desde el panel de archivos, o poner `USAR_DRIVE = False`
`cargar_modelo` falla con un error poco claro	No se reinició la sesión tras la celda 1	Reiniciar la sesión y seguir desde la celda 2
`No hay GPU`	El entorno no tiene GPU	Elegir T4 GPU y empezar desde la celda 1
La búsqueda se interrumpe	Desconexión o límite de GPU de Colab	Volver a ejecutar: con el caché en Drive retoma sin recalcular

5.3 Supuestos

- Cada consulta tiene el mismo costo y su resultado es binario (todo o nada).
- Todas las configuraciones se comparan sobre las mismas 100 instancias de búsqueda.
- El orden inicial de ranuras usa una hipótesis ranura → familias basada en la documentación del curso; las trazas la confirmaron solo en parte.

5.4 Limitaciones y componentes aún no desarrollados

- El resultado es un **óptimo local**: ningún cambio de una sola ranura lo mejora, pero el techo reportado por el curso es cercano al 24 %.
- La poda es permanente y su umbral (3 fallos) es bajo frente al ruido de 100 instancias.
- La diferencia entre 14 % y 12 % equivale a 2 instancias y está dentro del error de muestreo.
- Pendiente inmediato: agregar a `pruebas/` el caso de prueba base que compartirá el monitor y casos propios construidos a partir de él.

- Pendientes para las siguientes entregas: algoritmo exacto de referencia (backtracking con poda) y Successive Halving sobre instancias sintéticas de tamaño creciente, comparación de reparto uniforme frente a adaptativo, y validación experimental del crecimiento de T(n) contra n.